

Lab 19 - StatefulSets, Headless Services and Stable Identity

Field	Value
Lab id	Lab 19
Course	TGS-2023039343 Kubernetes and Cloud Native Associate (KCNA) Training v6.0
Day / Topic	Day 4 Storage
Duration	50 minutes
Namespace	kcna-lab19
Maps to	LO4 Prepare a technical blueprint for a Kubernetes-based solution for security and storage A4 Prepare a technical blueprint for a solution in a given area K6 Technical blueprint design and construction process

1. Objective

By the end of this lab you will be able to:

1. State the four guarantees a StatefulSet gives that a Deployment does not - stable ordinal names, stable per-Pod DNS, stable per-Pod storage, and ordered lifecycle - and choose correctly between the two controllers.
2. Create a headless Service (clusterIP: None) and explain what CoreDNS publishes for it that it does not publish for a ClusterIP Service.
3. Resolve and address an individual replica at <pod>.<svc>.<ns>.svc.cluster.local.
4. Use volumeClaimTemplates to have the controller create one PersistentVolumeClaim per ordinal, and demonstrate that a deleted Pod is reattached to *its own* original volume.
5. Observe ordered creation (0 → 1 → 2) and reverse-ordered termination (2 → 1 → 0), and switch that behaviour off with podManagementPolicy: Parallel.
6. Explain why PVCs created from volumeClaimTemplates are deliberately *not* deleted with the StatefulSet, and clean them up safely inside the lab namespace.
7. Diagnose an NXDOMAIN caused by a missing headless Service, using nslookup and kubectl get endpointslices.

2. Prerequisites

* Single-node kind cluster, Kubernetes v1.30+, with the default standard StorageClass present (verify with kubectl get sc).

* Labs 17 and 18 completed. You need to be comfortable with PVC binding and WaitForFirstConsumer before you meet volumeClaimTemplates.

* Terminal at this lab folder:

```
cd courseware/labs/lab-19-statefulset-headless
pwd
```

```
.../courseware/labs/lab-19-statefulset-headless
```

```
---
```

3. Scenario

Kallang Freight's TrackLane platform keeps a **depot ledger**: an append-only record of stock movements, sharded three ways so that each of the three physical depots owns its own slice.

The platform team originally shipped it as a Deployment with three replicas and one shared PVC. Two things went wrong in the first week:

1. All three replicas wrote to the same volume and corrupted each other. (You now know why: a `ReadWriteOnce` volume permits multiple Pods **on the same node** to mount it read-write. It is not a lock.)
2. The reconciliation job needs to talk to shard 1 specifically to replay a movement. With a Deployment there is no way to name a particular replica - the Service load-balances, and the Pod names are random hashes that change on every rollout.

The fix is a StatefulSet: three members with permanent names `depot-ledger-0`, `-1`, `-2`, each with its own volume, each individually addressable by DNS. This lab builds it and then proves each guarantee holds.

```
---
```

4. Step-by-step procedure

Step 1 - Namespace and datasets

```
kubectl apply -f manifests/00-namespace.yaml
```

```
namespace/kcna-lab19 created
```

Two datasets drive this lab. First, the roster that maps an ordinal to a depot:

```
cat data/depot-roster.csv
```

```
ordinal,depot_code,depot_name,region,berths,ledger_shard
0,depot-a,Kallang Basin Depot,SG-Central,4,shard-0
1,depot-b,Tuas Mega Depot,SG-West,9,shard-1
2,depot-c,Changi Air Depot,SG-East,6,shard-2
```

Second, the ledger itself. Count the entries per shard - these are the numbers each Pod must independently produce:

```
for s in shard-0 shard-1 shard-2; do
  printf '%s = %s entries\n' "$s" "$$(grep -c ",$s," data/ledger-entries.csv)"
done
```

```
shard-0 = 9 entries
shard-1 = 8 entries
shard-2 = 7 entries
```

Load both into one ConfigMap:

```
kubectl -n kcna-lab19 create configmap tracklane-ledger-seed \
  --from-file=depot-roster.csv=data/depot-roster.csv \
  --from-file=ledger-entries.csv=data/ledger-entries.csv
```

```
configmap/tracklane-ledger-seed created
```

Step 2 - Create the headless Service **first**

Order matters. The StatefulSet controller will not create this for you, and will not complain if it is absent.

```
kubectl apply -f manifests/01-headless-service.yaml
```

```
service/depot-ledger created
```

```
kubectl -n kcna-lab19 get service depot-ledger
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
depot-ledger	ClusterIP	None	<none>	80/TCP	5s

CLUSTER-IP: None. That single value is what makes this Service headless:

	ClusterIP Service	Headless Service (clusterIP: None)
Virtual IP allocated	Yes	No
kube-proxy rules programmed	Yes (iptables/IPVS/nftables DNAT)	No
DNS for the Service name	One A record - the VIP	One A record per Ready backing Pod
Per-Pod DNS record	No	Yes, when a StatefulSet sets serviceName
Load balancing	kube-proxy, per connection	Client's choice (or DNS round-robin)
Typical use	Stateless web tier	Databases, queues, quorum systems, peer discovery

Step 3 - Create the StatefulSet and watch it come up in order

Before applying, read the five identity-defining fields:

```
grep -n 'serviceName\|replicas:\|podManagementPolicy\|whenDeleted\|whenScaled' manifests/03-statefulset.yaml
```

```
30: serviceName: depot-ledger
31: replicas: 3
34: podManagementPolicy: OrderedReady
47:   whenDeleted: Retain
48:   whenScaled: Retain
```

Open a second terminal and start a watch:

```
kubectl -n kcna-lab19 get pods -w
```

Back in the first terminal:

```
kubectl apply -f manifests/03-statefulset.yaml
```

```
statefulset.apps/depot-ledger created
```

The watch terminal shows creation strictly in ordinal order, each Pod waiting for its predecessor to be Ready:

NAME	READY	STATUS	RESTARTS	AGE
depot-ledger-0	0/1	Pending	0	0s
depot-ledger-0	0/1	Init:0/1	0	2s
depot-ledger-0	0/1	PodInitializing	0	6s
depot-ledger-0	1/1	Running	0	9s
depot-ledger-1	0/1	Pending	0	0s
depot-ledger-1	0/1	Init:0/1	0	2s
depot-ledger-1	1/1	Running	0	8s
depot-ledger-2	0/1	Pending	0	0s
depot-ledger-2	0/1	Init:0/1	0	2s
depot-ledger-2	1/1	Running	0	8s

Stop the watch with Ctrl-C. Then:

```
kubectl -n kcna-lab19 rollout status statefulset/depot-ledger --timeout=240s
```

```
partitioned roll out complete: 3 new pods have been updated...
```

```
kubectl -n kcna-lab19 get statefulset,pods
```

NAME	READY	AGE
statefulset.apps/depot-ledger	3/3	52s

NAME	READY	STATUS	RESTARTS	AGE
pod/depot-ledger-0	1/1	Running	0	52s
pod/depot-ledger-1	1/1	Running	0	43s
pod/depot-ledger-2	1/1	Running	0	34s

Look at the Pod names. depot-ledger-0, -1, -2 - no random hash. Compare with a Deployment, whose Pods are named <deploy>-<replicaset-hash>-<pod-hash> and change identity on every rollout.

Step 4 - One PersistentVolumeClaim per ordinal

```
kubectl -n kcna-lab19 get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS
ledger-depot-ledger-0	Bound	pvc-3a71c0e5-9b28-4d16-8f04-c7e2b915d6a3	128Mi	RWO
standard	<unset>	61s		
ledger-depot-ledger-1	Bound	pvc-d0c48f92-16ba-4e37-95a1-2f8b60e4c771	128Mi	RWO
standard	<unset>	52s		
ledger-depot-ledger-2	Bound	pvc-7e5b1c34-a802-49df-b6c8-51d0937fae28	128Mi	RWO
standard	<unset>	43s		

You did not write these. The StatefulSet controller created one PVC per ordinal from the single volumeClaimTemplates entry, using the naming rule:

```
<volumeClaimTemplate.metadata.name>-<statefulset.metadata.name>-<ordinal>
ledger - depot-ledger - 0
```

Three distinct claims, three distinct dynamically provisioned PVs. This is precisely what a Deployment cannot do - its Pod template names one PVC, and every replica mounts that same one.

Confirm each Pod is wired to its own claim:

```
kubectl -n kcna-lab19 get pods -o custom-columns=\
'POD:.metadata.name,CLAIM:.spec.volumes[?(@.name=="ledger")].persistentVolumeClaim.claimName'
```

POD	CLAIM
depot-ledger-0	ledger-depot-ledger-0
depot-ledger-1	ledger-depot-ledger-1
depot-ledger-2	ledger-depot-ledger-2

And that each initContainer picked up a different shard:

```
for i in 0 1 2; do
  printf 'depot-ledger-%s: ' "$i"
  kubectl -n kcna-lab19 logs "depot-ledger-$i" -c seed-shard
done
```

```
depot-ledger-0: ordinal=0 shard=shard-0 entries=9 mounts=1
depot-ledger-1: ordinal=1 shard=shard-1 entries=8 mounts=1
depot-ledger-2: ordinal=2 shard=shard-2 entries=7 mounts=1
```

9, 8, 7 - exactly the per-shard counts you computed from data/ledger-entries.csv in Step 1. Each replica read the same ConfigMap, derived a *different* answer from its *own ordinal*, and wrote it to its *own* volume.

Step 5 - Address an individual replica by DNS

Start the in-cluster client:

```
kubectl apply -f manifests/04-ledger-client.yaml
kubectl -n kcna-lab19 wait --for=condition=Ready pod/ledger-client --timeout=90s
```

```
pod/ledger-client created
pod/ledger-client condition met
```

Resolve the headless Service name:

```
kubectl -n kcna-lab19 exec ledger-client -- \
nslookup depot-ledger.kcna-lab19.svc.cluster.local
```

```
Server:10.96.0.10
Address:10.96.0.10:53

Name:depot-ledger.kcna-lab19.svc.cluster.local
Address: 10.244.0.52
Name:depot-ledger.kcna-lab19.svc.cluster.local
Address: 10.244.0.54
Name:depot-ledger.kcna-lab19.svc.cluster.local
Address: 10.244.0.56
```

Three addresses for one name - the Pod IPs themselves. No VIP anywhere. Now resolve a single member:

```
kubectl -n kcna-lab19 exec ledger-client -- \
  nslookup depot-ledger-1.depot-ledger.kcna-lab19.svc.cluster.local
```

```
Server:10.96.0.10
Address:10.96.0.10:53

Name:depot-ledger-1.depot-ledger.kcna-lab19.svc.cluster.local
Address: 10.244.0.54
```

One name, one address, permanently attached to ordinal 1. Decompose it:

```
depot-ledger-1 . depot-ledger . kcna-lab19 . svc . cluster.local
pod name      service name  namespace      cluster domain
```

Now **use** it - fetch the ledger page from a specific replica:

```
kubectl -n kcna-lab19 exec ledger-client -- \
  wget -qO- http://depot-ledger-1.depot-ledger.kcna-lab19.svc.cluster.local/
```

```
<h1>TrackLane depot ledger</h1>
<p>pod: depot-ledger-1</p>
<p>ordinal: 1</p>
<p>depot_code: depot-b</p>
<p>depot_name: Tuas Mega Depot</p>
<p>region: SG-West</p>
<p>ledger_shard: shard-1</p>
<p>shard_entries: 8</p>
<p>volume_mount_count: 1</p>
```

Loop over all three to see three genuinely different answers:

```
for i in 0 1 2; do
  kubectl -n kcna-lab19 exec ledger-client -- \
    wget -qO- "http://depot-ledger-$i.depot-ledger.kcna-lab19.svc.cluster.local/" \
    | grep -E 'pod:|depot_name:|shard_entries:'
done
```

```
<p>pod: depot-ledger-0</p>
<p>depot_name: Kallang Basin Depot</p>
<p>shard_entries: 9</p>
<p>pod: depot-ledger-1</p>
<p>depot_name: Tuas Mega Depot</p>
<p>shard_entries: 8</p>
<p>pod: depot-ledger-2</p>
<p>depot_name: Changi Air Depot</p>
<p>shard_entries: 7</p>
```

This is the reconciliation job's problem solved. It can now say "replay into shard 1" and reach exactly one process.

Step 6 - Contrast with a normal ClusterIP Service

```
kubectl apply -f manifests/02-clusterip-service.yaml
```

```
service/depot-ledger-vip created
```

```
kubectl -n kcna-lab19 get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
depot-ledger	ClusterIP	None	<none>	80/TCP	6m
depot-ledger-vip	ClusterIP	10.96.184.207	<none>	80/TCP	4s

```
kubectl -n kcna-lab19 exec ledger-client -- \
  nslookup depot-ledger-vip.kcna-lab19.svc.cluster.local
```

```
Server: 10.96.0.10
Address: 10.96.0.10:53
```

```
Name: depot-ledger-vip.kcna-lab19.svc.cluster.local
Address: 10.96.184.207
```

One address, and it is not a Pod. It is the VIP; kube-proxy DNATs it to a backend of its choosing. Prove that you cannot control which replica answers:

```
for n in 1 2 3 4 5 6; do
  kubectl -n kcna-lab19 exec ledger-client -- \
    wget -qO- http://depot-ledger-vip.kcna-lab19.svc.cluster.local/ | grep 'pod:'
done
```

```
<p>pod: depot-ledger-2</p>
<p>pod: depot-ledger-0</p>
<p>pod: depot-ledger-2</p>
<p>pod: depot-ledger-1</p>
<p>pod: depot-ledger-0</p>
<p>pod: depot-ledger-1</p>
```

Your sequence will differ and is not round-robin - kube-proxy in iptables mode picks a backend at random per connection. That unpredictability is exactly right for a stateless tier and exactly wrong for a ledger shard.

Both Services select the same Pods, which you can see in the EndpointSlices:

```
kubectl -n kcna-lab19 get endpointslices \
  -o custom-columns='SLICE:.metadata.name,SERVICE:.metadata.labels.kubernetes\.io/service-name,
  ADDRESSES:.endpoints[*].addresses'
```

SLICE	SERVICE	ADDRESSES
depot-ledger-jm4x9	depot-ledger	10.244.0.52,10.244.0.54,10.244.0.56
depot-ledger-vip-t7q2b	depot-ledger-vip	10.244.0.52,10.244.0.54,10.244.0.56

discovery.k8s.io/v1 EndpointSlice is the current API for Service backends. The legacy v1 Endpoints API was deprecated in Kubernetes v1.33 - `kubectl get endpointslices` is what you should reach for now.

Step 7 - Prove identity survives Pod deletion

Record the current state of ordinal 1:

```
kubectl -n kcna-lab19 get pod depot-ledger-1 -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
depot-ledger-1	1/1	Running	0	9m	10.244.0.54	kcna-control-plane

Delete it:

```
kubectl -n kcna-lab19 delete pod depot-ledger-1 --wait=true
```

```
pod "depot-ledger-1" deleted
```

```
kubectl -n kcna-lab19 wait --for=condition=Ready pod/depot-ledger-1 --timeout=180s
kubectl -n kcna-lab19 get pod depot-ledger-1 -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
depot-ledger-1	1/1	Running	0	14s	10.244.0.61	kcna-control-plane

Compare carefully:

Property	Before	After	Stable?
Pod name	depot-ledger-1	depot-ledger-1	Yes
Pod IP	10.244.0.54	10.244.0.61	No - and it does not matter
DNS name	depot-ledger-1.depot-ledger...	same	Yes
PVC	ledger-depot-ledger-1	same	Yes
Pod UID	old	new	No - it is a different Pod object

The DNS record followed the ordinal to the new IP:

```
kubectl -n kcna-lab19 exec ledger-client -- \
nslookup depot-ledger-1.depot-ledger.kcna-lab19.svc.cluster.local | tail -2
```

```
Name: depot-ledger-1.depot-ledger.kcna-lab19.svc.cluster.local
Address: 10.244.0.61
```

And - the decisive evidence - the volume was reattached, not recreated:

```
kubectl -n kcna-lab19 logs depot-ledger-1 -c seed-shard
```

```
ordinal=1 shard=shard-1 entries=8 mounts=2
```

```
kubectl -n kcna-lab19 exec depot-ledger-1 -c web -- cat /usr/share/nginx/html/index.html |
grep volume_mount_count
```

```
<p>volume_mount_count: 2</p>
```

mounts=2. The mounts.log file on the PVC now has two lines: one written by the Pod you deleted, one by its replacement. The replacement Pod found the old Pod's data waiting for it. Confirm:

```
kubectl -n kcna-lab19 exec depot-ledger-1 -c web -- \
sh -c 'cat /usr/share/nginx/html/mounts.log'
```

```
mounted depot-ledger-1 2026-09-05T06:31:12Z
mounted depot-ledger-1 2026-09-05T06:40:47Z
```

Two different timestamps, one volume. That is stable per-Pod storage.

Step 8 - Ordered termination and scaling

Scale down to 1 and watch the order. In your second terminal:

```
kubectl -n kcna-lab19 get pods -w
```

Then:

```
kubectl -n kcna-lab19 scale statefulset depot-ledger --replicas=1
```

```
statefulset.apps/depot-ledger scaled
```

The watch shows reverse ordinal order - highest first:

```
depot-ledger-2    1/1    Terminating    0        14m
depot-ledger-2    0/1    Terminating    0        14m
depot-ledger-2    0/1    Completed       0        14m
depot-ledger-1    1/1    Terminating    0        4m
depot-ledger-1    0/1    Terminating    0        4m
depot-ledger-1    0/1    Completed       0        4m
```

Ctrl-C the watch.

```
kubectl -n kcna-lab19 get pods
```

NAME	READY	STATUS	RESTARTS	AGE
depot-ledger-0	1/1	Running	0	15m
ledger-client	1/1	Running	0	9m

Why reverse order matters. In a quorum system (etcd, ZooKeeper, a Raft group) the members joined in ascending order. Removing the *lowest* ordinal first would take out the bootstrap/seed member while the others still depend on it. Removing the highest first preserves quorum at every intermediate step.

Now the critical observation:

```
kubectl -n kcna-lab19 get pvc
```

NAME	STORAGECLASS	STATUS	VOLUME	CAPACITY	ACCESS
MODES		VOLUMEATTRIBUTESCLASS	AGE		
ledger-depot-ledger-0	standard	Bound	pvc-3a71c0e5-9b28-4d16-8f04-c7e2b915d6a3	128Mi	RWO
		<unset>	15m		
ledger-depot-ledger-1	standard	Bound	pvc-d0c48f92-16ba-4e37-95a1-2f8b60e4c771	128Mi	RWO
		<unset>	15m		
ledger-depot-ledger-2	standard	Bound	pvc-7e5b1c34-a802-49df-b6c8-51d0937fae28	128Mi	RWO
		<unset>	15m		

Still three PVCs, though only one Pod remains. This is `persistentVolumeClaimRetentionPolicy.whenScaled: Retain` - the default - and it is deliberate: scaling down is usually temporary, and silently destroying a database replica's data because someone typed `--replicas=1` would be catastrophic.

Scale back up and watch shard 2's data come back untouched:

```
kubectl -n kcna-lab19 scale statefulset depot-ledger --replicas=3
kubectl -n kcna-lab19 rollout status statefulset/depot-ledger --timeout=240s
kubectl -n kcna-lab19 logs depot-ledger-2 -c seed-shard
```

```
statefulset.apps/depot-ledger scaled
partitioned roll out complete: 3 new pods have been updated...
ordinal=2 shard=shard-2 entries=7 mounts=2
```

`mounts=2` for ordinal 2 as well - it re-adopted the volume it had before the scale-down.

The field to know. `spec.persistentVolumeClaimRetentionPolicy` has two independent keys:

>

| Key | Retain (default) | Delete |

|---|---|---|

| whenDeleted | PVCs survive deletion of the StatefulSet | PVCs are garbage-collected with the StatefulSet |

| whenScaled | PVCs of removed ordinals survive | PVCs of removed ordinals are deleted on scale-down |

>

The field reached beta and default-on in v1.27 and GA in v1.32. On an older cluster it is ignored and you get `Retain/Retain` regardless - which is why explicitly writing the defaults, as this manifest does, is good practice.

Step 9 - podManagementPolicy: Parallel

`OrderedReady` costs startup time proportional to the number of replicas. When members are independent, use `Parallel`.

```
kubectl apply -f manifests/06-parallel-indexer.yaml
```

```
service/lane-indexer created
statefulset.apps/lane-indexer created
```

```
kubectl -n kcna-lab19 get pods -l app.kubernetes.io/name=lane-indexer
```

NAME	READY	STATUS	RESTARTS	AGE
lane-indexer-0	0/1	ContainerCreating	0	2s
lane-indexer-1	0/1	ContainerCreating	0	2s
lane-indexer-2	0/1	ContainerCreating	0	2s

All three at AGE 2s - created simultaneously, not sequentially. Compare Step 3, where the ages were staggered by ~9 seconds each.

The identity guarantees are unchanged:

```
kubectl -n kcna-lab19 wait --for=condition=Ready pod -l app.kubernetes.io/name=lane-indexer --
timeout=180s
kubectl -n kcna-lab19 exec ledger-client -- \
  wget -q0- http://lane-indexer-2.lane-indexer.kcna-lab19.svc.cluster.local/
```

```
pod/lane-indexer-0 condition met
pod/lane-indexer-1 condition met
pod/lane-indexer-2 condition met
lane-indexer ordinal 2 ready
```

Note also that lane-indexer has no volumeClaimTemplates at all:

```
kubectl -n kcna-lab19 get pvc -l app.kubernetes.io/name=lane-indexer
```

```
No resources found in kcna-lab19 namespace.
```

A StatefulSet is fundamentally about identity, not storage. Per-ordinal volumes are the most common reason to want that identity, not a requirement of it.

Step 10 - StatefulSet versus Deployment: the decision

Question	Deployment	StatefulSet
Pod names	web-6d4cf56db6-x9tzq - random, changes every rollout	depot-ledger-1 - ordinal, permanent
Individually addressable by DNS?	No	Yes, via a headless Service
Storage	All replicas share whatever PVC the template names	One PVC per ordinal, auto-created
Startup / shutdown order	Arbitrary, concurrent	Ordered (OrderedReady) or concurrent (Parallel)
Rolling update order	Governed by maxSurge / maxUnavailable	Highest ordinal first; gated by partition
Scale-down removes	An arbitrary replica	Always the highest ordinal
Replicas interchangeable?	Yes - that is the point	No - each has a role
Use it for	Web tiers, APIs, workers, anything stateless	Databases, brokers, quorum systems, sharded stores

Choose a Deployment unless you can name a specific guarantee above that you need. StatefulSets are slower to roll, harder to debug, and leave PVCs behind. In cloud-native practice, the strongest advice is stronger still: for production databases, prefer a managed service or a mature Operator (CloudNativePG, Strimzi, Vitess) that wraps a StatefulSet

with backup, failover and version-upgrade logic. A raw StatefulSet gives you identity and storage; it gives you nothing about consensus, backup or promotion.

5. Verification

```
bash verification/checks.sh
```

```
== Lab 19 verification - namespace kcna-lab19 ==
[PASS] namespace kcna-lab19 exists
[PASS] service/depot-ledger is headless (clusterIP: None)
[PASS] service/depot-ledger-vip has an allocated ClusterIP
[PASS] statefulset/depot-ledger has serviceName=depot-ledger
[PASS] statefulset/depot-ledger reports 3/3 ready replicas
[PASS] pods are named by ordinal: depot-ledger-0, -1, -2
[PASS] one PVC exists per ordinal from volumeClaimTemplates
[PASS] each pod mounts its own ordinal's PVC
[PASS] shard entry counts 9/8/7 match data/ledger-entries.csv
[PASS] per-pod DNS resolves depot-ledger-1.depot-ledger.kcna-lab19.svc.cluster.local
[PASS] headless DNS returns 3 pod addresses, VIP DNS returns 1
[PASS] depot-ledger-1 reattached its original volume (mount count >= 2)
[PASS] persistentVolumeClaimRetentionPolicy is Retain/Retain
[PASS] statefulset/depot-ledger uses podManagementPolicy=Parallel
[PASS] no container in kcna-lab19 requests privileged: true
-----
15 passed, 0 failed
```

Full transcript: verification/expected-output.md.

6. Failure injection

The ticket: **"PLAT-4488 - the ghost-ledger StatefulSet is green in the dashboard but the reconciliation job cannot reach it."**

```
kubectl apply -f manifests/05-broken-no-headless-service.yaml
kubectl -n kcna-lab19 rollout status statefulset/ghost-ledger --timeout=180s
```

```
statefulset.apps/ghost-ledger created
partitioned roll out complete: 1 pods have been updated...
```

Everything looks healthy:

```
kubectl -n kcna-lab19 get statefulset ghost-ledger
kubectl -n kcna-lab19 get pod ghost-ledger-0
```

NAME	READY	AGE
ghost-ledger	1/1	31s

NAME	READY	STATUS	RESTARTS	AGE
ghost-ledger-0	1/1	Running	0	31s

1/1 Running, no events, no warnings. Now try to use the identity the StatefulSet promised:

```
kubectl -n kcna-lab19 exec ledger-client -- \
  nslookup ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local
```

```
Server:10.96.0.10
Address:10.96.0.10:53
```

```
** server can't find ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local: NXDOMAIN
command terminated with exit code 1
```

```
kubectl -n kcna-lab19 exec ledger-client -- \
  wget -T 5 -qO- http://ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local/
```

```
wget: bad address 'ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local'
command terminated with exit code 1
```

Diagnosis, in order:

1. NXDOMAIN means the name does not exist - this is a naming/DNS fault, not a connectivity or policy fault. A NetworkPolicy block or a crashed Pod would give a timeout or a connection refusal, not NXDOMAIN.
2. The per-Pod name has the form <pod>.<serviceName>.<ns>.svc.cluster.local. The middle label is ghost-ledger. Is there a Service by that name?

```
kubectl -n kcna-lab19 get service ghost-ledger
```

```
Error from server (NotFound): services "ghost-ledger" not found
```

3. There it is. Confirm the StatefulSet is asking for a Service that does not exist:

```
kubectl -n kcna-lab19 get statefulset ghost-ledger -o jsonpath='{.spec.serviceName}{"\n"}'
```

```
ghost-ledger
```

4. Corroborate from the other side - no Service means no EndpointSlice, so nothing ever gets published to DNS:

```
kubectl -n kcna-lab19 get endpointslices -l kubernetes.io/service-name=ghost-ledger
```

```
No resources found in kcna-lab19 namespace.
```

Compare with the working set, which has one:

```
kubectl -n kcna-lab19 get endpointslices -l kubernetes.io/service-name=depot-ledger
```

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
depot-ledger-jm4x9	IPv4	80	10.244.0.52,10.244.0.61,10.244.0.66	28m

Root cause. spec.serviceName is a *reference by name only*. Kubernetes performs no validation that the named Service exists, is headless, or selects these Pods. There is no event, no warning and no status condition - because from the StatefulSet controller's point of view nothing is wrong: it created the Pods it was asked for and they are Ready.

Fix. Create the missing headless Service with a selector matching the Pods:

```
kubectl apply -f - <<'YAML'
apiVersion: v1
kind: Service
metadata:
  name: ghost-ledger
  namespace: kcna-lab19
spec:
  clusterIP: None
  selector:
    app.kubernetes.io/name: ghost-ledger
  ports:
    - name: http
      port: 80
      targetPort: http
      protocol: TCP
YAML
```

```
service/ghost-ledger created
```

```
sleep 10
kubectl -n kcna-lab19 exec ledger-client -- \
  nslookup ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local | tail -2
```

```
Name: ■ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local
Address: 10.244.0.71
```

Resolved. Nothing about the StatefulSet changed; the missing half of the pair simply appeared.

Two near-miss variants to recognise:

Variant	Symptom	Why
Service exists but has a ClusterIP (not headless)	The service name resolves to the VIP, but <pod>.<svc>... is still NXDOMAIN	Per-Pod records are only published for headless Services
Service exists and is headless but its selector does not match	Both names NXDOMAIN; the Service shows <none> endpoints	No EndpointSlice entries means nothing to publish

Remove the injection:

```
kubectl -n kcna-lab19 delete statefulset ghost-ledger --ignore-not-found
kubectl -n kcna-lab19 delete service ghost-ledger --ignore-not-found
```

```
statefulset.apps "ghost-ledger" deleted
service "ghost-ledger" deleted
```

7. Troubleshooting

Symptom	Likely cause	Diagnostic command	Fix
<pod>.<svc>.<ns>.svc.cluster.local returns NXDOMAIN	The Service named by spec.serviceName does not exist, or is not headless	kubectl -n kcna-lab19 get svc <serviceName> -o jsonpath='{.spec.clusterIP}'	Create the Service with clusterIP: None and a matching selector
Service name resolves but per-Pod names do not	The Service has a ClusterIP; only headless Services get per-Pod records	kubectl -n kcna-lab19 get svc -o wide	Recreate the Service with clusterIP: None (the field is immutable)
Headless Service resolves to nothing; endpointslices is empty	The Service selector does not match the Pod template labels	kubectl -n kcna-lab19 get endpointslices -l kubernetes.io/service-name=<svc>	Align service.spec.selector with statefulset.spec.template.metadata.labels
depot-ledger-1 never starts; -0 is Pending	OrderedReady blocks on the predecessor. The real fault is on ordinal 0	kubectl -n kcna-lab19 describe pod depot-ledger-0	Fix ordinal 0 (usually an unbound PVC or a failing readiness probe)
Pod Pending, event pod has unbound immediate PersistentVolumeClaims	The per-ordinal PVC has not bound. On kind this is normal for a few seconds under WaitForFirstConsumer	kubectl -n kcna-lab19 describe pvc ledger-<sts>-<n>	Wait; if it persists, check the StorageClass name in volumeClaimTemplates
PVCs still present after deleting the StatefulSet	persistentVolumeClaimRetentionPolicy.whenDeleted: Retain - the default and deliberate	kubectl -n kcna-lab19 get pvc	Delete the PVCs explicitly by name, or set whenDeleted: Delete if the data is genuinely disposable
Scaling down destroyed replica data	whenScaled: Delete was set	kubectl -n kcna-lab19 get sts <name> -o jsonpath='{.spec.persistentVolumeClaimRetentionPolicy}'	Use whenScaled: Retain for anything with real state. There is no undo
Editing volumeClaimTemplates is rejected	Almost all of a StatefulSet's spec is immutable after creation	kubectl -n kcna-lab19 replace --force -f manifests/03-statefulset.yaml	Delete the StatefulSet with --cascade=orphan, change it, re-apply. The PVCs are retained
Rolling update stalls part-way through	updateStrategy.rollingUpdate.partition is above 0, so low ordinals are held back	kubectl -n kcna-lab19 get sts <name> -o jsonpath='{.spec.updateStrategy}'	Lower partition to 0 once the canary ordinal is healthy

8. Cleanup

The point of this section is that a StatefulSet does not clean up after itself.

Delete the workloads first, then look at what is left:

```
kubectl -n kcna-lab19 delete statefulset depot-ledger lane-indexer --ignore-not-found
```

```
statefulset.apps "depot-ledger" deleted
statefulset.apps "lane-indexer" deleted
```

```
kubectl -n kcna-lab19 get pods
```

NAME	READY	STATUS	RESTARTS	AGE
ledger-client	1/1	Running	0	41m

```
kubectl -n kcna-lab19 get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS
MODES STORAGECLASS VOLUMEATTRIBUTESCLASS AGE				
ledger-depot-ledger-0 standard	Bound <unset>	pvc-3a71c0e5-9b28-4d16-8f04-c7e2b915d6a3 44m	128Mi	RWO
ledger-depot-ledger-1 standard	Bound <unset>	pvc-d0c48f92-16ba-4e37-95a1-2f8b60e4c771 44m	128Mi	RWO
ledger-depot-ledger-2 standard	Bound <unset>	pvc-7e5b1c34-a802-49df-b6c8-51d0937fae28 44m	128Mi	RWO

All three claims survive. The StatefulSet is gone; its storage is not. This is whenDeleted: Retain, and it is a feature - it is what lets you delete and recreate a database StatefulSet without losing the database.

Remove them by explicit name, inside the lab namespace only:

```
kubectl -n kcna-lab19 delete pvc \
  ledger-depot-ledger-0 ledger-depot-ledger-1 ledger-depot-ledger-2
```

```
persistentvolumeclaim "ledger-depot-ledger-0" deleted
persistentvolumeclaim "ledger-depot-ledger-1" deleted
persistentvolumeclaim "ledger-depot-ledger-2" deleted
```

Never clean up a StatefulSet's storage with `kubectl delete pvc --all`. Name the claims. On a shared cluster that flag has destroyed production databases.

The dynamically provisioned PVs are reclaimed automatically, because standard uses reclaimPolicy: Delete:

```
kubectl get pv | grep kcna-lab19 || echo "all dynamically provisioned volumes reclaimed"
```

```
all dynamically provisioned volumes reclaimed
```

Now remove the namespace, which takes everything else with it:

```
kubectl delete namespace kcna-lab19 --wait=true
```

```
namespace "kcna-lab19" deleted
```

```
kubectl get namespace kcna-lab19
```

```
Error from server (NotFound): namespaces "kcna-lab19" not found
```

This lab created no cluster-scoped objects. The only PVs involved were created and reclaimed by the standard StorageClass on your behalf. Deleting the namespace on its own would also have removed the PVCs - the explicit delete above is there to make the retention behaviour visible, which is the lesson.

9. What you learned

* A StatefulSet gives four guarantees a Deployment does not: stable ordinal names, stable per-Pod DNS, stable per-Pod storage, and ordered lifecycle. You need a StatefulSet only when you need one of those by name.

* A headless Service (clusterIP: None) has no VIP and no kube-proxy rules. CoreDNS publishes one address per Ready backing Pod, and - when a StatefulSet names it in serviceName - one record per Pod at <pod>.<svc>.<ns>.svc.cluster.local.

* volumeClaimTemplates creates one PVC per ordinal, named <template>-<statefulset>-<ordinal>. You proved a deleted Pod is reattached to its original volume by watching the mount counter go from 1 to 2 while the Pod IP changed.

* Identity is not the IP. depot-ledger-1 kept its name, its DNS record and its data across deletion; only its IP and UID changed. Every stateful protocol should address the name, never the address.

* podManagementPolicy: OrderedReady starts 0 → 1 → 2 and terminates 2 → 1 → 0, which is what preserves quorum in consensus systems. Parallel trades that ordering for startup speed and keeps every other guarantee.

* PVCs are deliberately not garbage-collected. persistentVolumeClaimRetentionPolicy (whenDeleted / whenScaled, defaulting to Retain/Retain) is the field that governs it. Cleaning up means naming the claims - never --all.

* spec.serviceName is unvalidated. A missing or non-headless Service produces a perfectly healthy 1/1 Running StatefulSet whose only symptom is NXDOMAIN. nslookup plus kubectl get endpointslices is the diagnostic pair.

* A raw StatefulSet supplies identity and storage and nothing else. Production data systems want an Operator or a managed service on top.

Carry into Lab 20: everything you have built on Day 4 is stored as an object in etcd, and scheduled by controllers you have not yet met. Lab 20 opens the control plane.

10. Further reading

* Kubernetes documentation - *StatefulSets*:

<<https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/>>

* Kubernetes documentation - *StatefulSet Basics* tutorial:

<<https://kubernetes.io/docs/tutorials/stateful-application/basic-stateful-set/>>

* Kubernetes documentation - *Headless Services*:

<<https://kubernetes.io/docs/concepts/services-networking/service/#headless-services>>

* Kubernetes documentation - *DNS for Services and Pods* (the record formats used in Step 5):

<<https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/>>

* Kubernetes documentation - *PersistentVolumeClaim retention* for StatefulSets:

<<https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/#persistentvolumeclaim-retention>>

* Kubernetes blog - *Kubernetes v1.33: Endpoints deprecation* (use EndpointSlice):

<<https://kubernetes.io/blog/2025/04/24/endpoints-deprecation/>>

* Kubernetes documentation - *Operator pattern*:

<<https://kubernetes.io/docs/concepts/extend-kubernetes/operator/>>

* CNCF KCNA Curriculum - *Container Orchestration → Storage* and *Kubernetes Fundamentals → Core Concepts*: <<https://github.com/cncf/curriculum>>